

QSign

How quantum-safe e-signatures work

A plain-language guide for owners, customers and security teams

What is in this guide

Part	What you will learn
1 The problem	Why today's digital signatures need replacing, and why it matters now
2 Five ideas	Fingerprints, keys, signatures, post-quantum, hybrid: no maths needed
3 The system	Which AWS services QSign uses and what each one does
4 Signing	Step by step, what happens when someone signs
5 Envelopes	Several signers, signing order and the evidence pack
6 Verifying	How anyone can check a signature, and what a failure means
7 Security	Who can do what, and how common attacks are stopped
8 The law	Where QSign stands in the USA, India and the EU
9 Running it	Deploying, onboarding a customer, plans and what comes next

In one sentence

QSign signs documents twice, once with a new quantum-safe method and once with today's trusted method, using keys locked inside AWS hardware, and keeps a tamper-proof record so anyone can later prove who signed what, and when.

The problem

Every digital signature in use today relies on maths that a large quantum computer will be able to break.

Almost all e-signatures (PDF signatures, Aadhaar eSign, DocuSign certificates, website certificates) use **RSA** or **elliptic-curve** cryptography. Their safety rests on maths problems that ordinary computers cannot solve in any reasonable time. In 1994 Peter Shor showed that a sufficiently large quantum computer could solve them quickly. Such a machine does not exist yet, but governments are planning for the day it does.

Why act now?

- **Contracts live a long time.** A property deed, loan or employment contract signed today may be disputed in 10, 20 or 30 years. If its signature can be forged by then, it proves nothing.
- **Deadlines are set.** The US NSA (CNSA 2.0) and NIST plan to phase out RSA and elliptic-curve signatures between 2030 and 2035. The EU asked member states to start moving in 2026 and to protect high-risk systems by 2030.
- **The replacement is ready.** In August 2024 NIST published **FIPS 204 (ML-DSA)**, a signature standard designed to resist quantum computers. AWS KMS has supported ML-DSA keys since 2025.

What QSign does about it

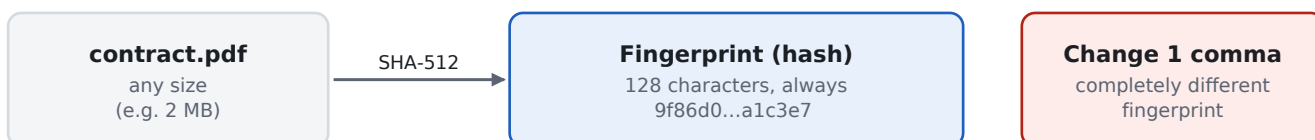
It signs with ML-DSA now, while still adding a classical ECDSA signature that today's tools trust. Customers get protection against tomorrow's attacks without losing compatibility today.

Five ideas you need

These five ideas explain everything QSign does. None of them need maths.

1. A fingerprint (hash)

A hash function turns any file into a short, fixed-length code. QSign uses **SHA-512**. It works like a fingerprint: unique to the file, and useless for rebuilding the file.



The same file always gives the same fingerprint. You cannot work backwards from the fingerprint to the file.

This is why QSign never needs your document. Your browser computes the fingerprint and only that is sent. If anyone later changes even one letter, the fingerprint no longer matches.

2 and 3. Keys and signatures

A signing key comes as a pair. The **private key** creates signatures and must stay secret. The **public key** checks them and is shared with everyone. A signature is the result of combining a fingerprint with the private key.

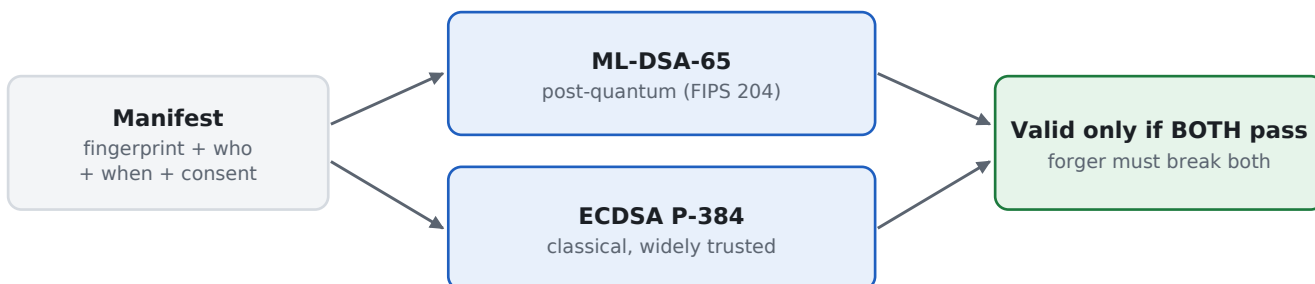


4. Post-quantum

ML-DSA (Module-Lattice Digital Signature Algorithm) is built on a different kind of maths (lattices) for which no fast quantum attack is known. QSign uses **ML-DSA-65**, the middle security level, roughly comparable to the strength of 192-bit classical security.

5. Hybrid

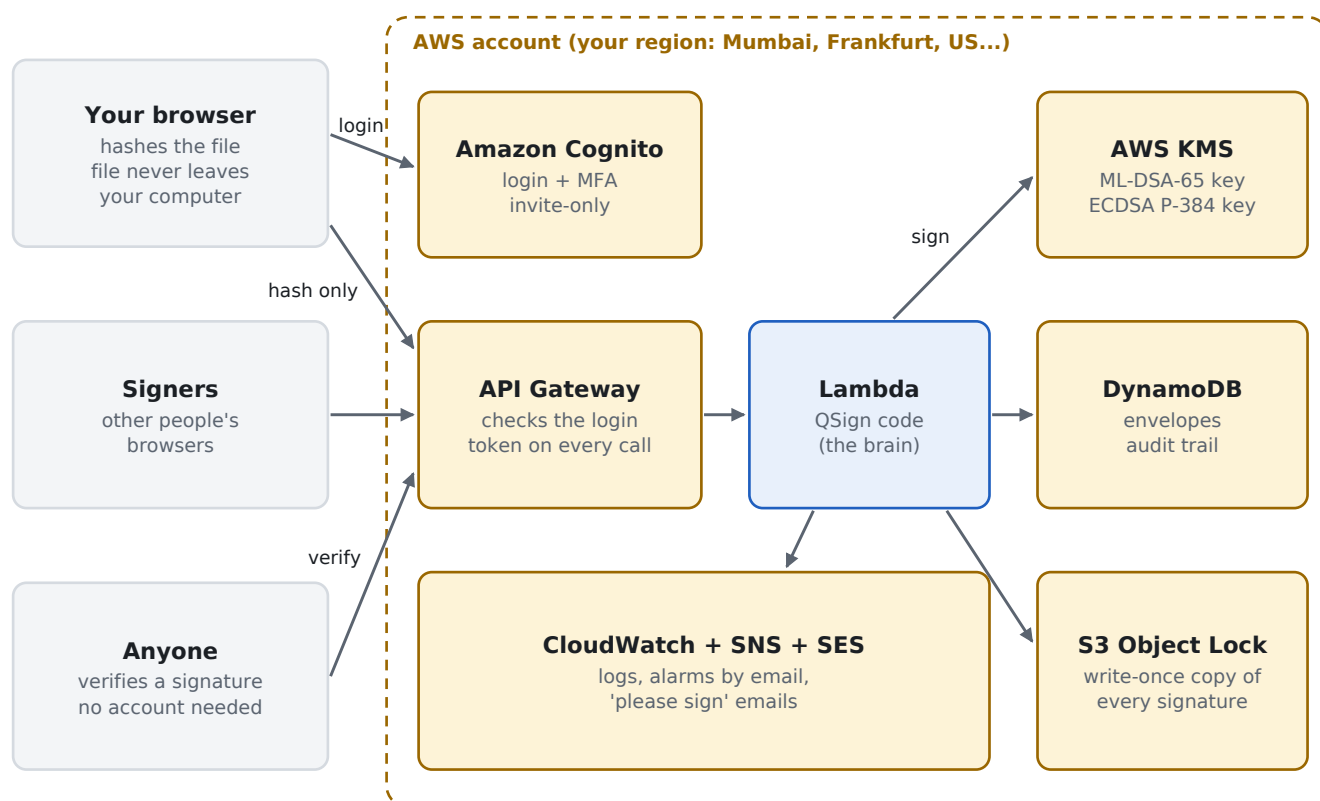
ML-DSA is new. Experts recommend using it alongside a proven classical signature during the transition, so a mistake in either one is not enough to forge a signature.



PART 3

The system

QSign is built only from managed AWS services, so there are no servers to patch and nothing to keep running overnight.

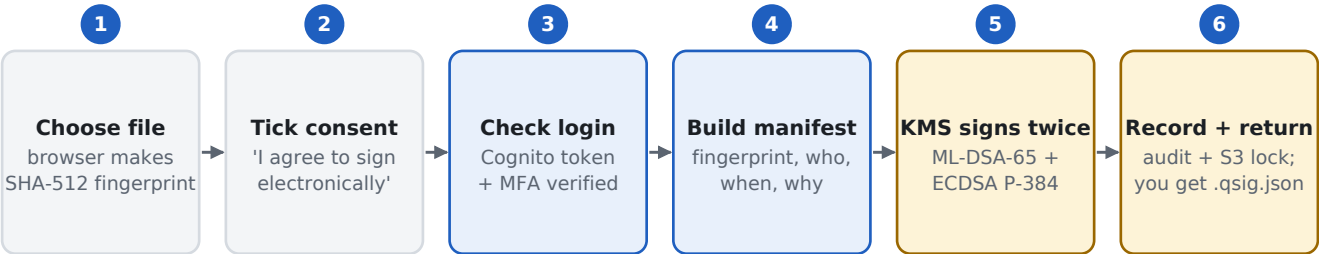


AWS service	Its job in QSign
AWS KMS	Holds the two signing keys in hardware security modules validated to FIPS 140-3 Level 3. Keys are created inside and can never be exported. QSign can only ask KMS to sign.
Amazon Cognito	User accounts. Invite-only, strong passwords, and an authenticator app (MFA) for everyone. Stores each user's organisation and whether they are an admin.
API Gateway	The front door. Checks the Cognito login token before any private request reaches QSign. Writes an access log line for every request and limits request rates.
AWS Lambda	Runs the QSign code only when a request arrives. Can also run as a container (ECS, EKS).
DynamoDB	Stores envelopes and the audit trail. Point-in-time recovery and deletion protection are on.
S3 Object Lock	Keeps a write-once copy of every signature and evidence pack. In compliance mode nobody, not even the account owner, can delete or alter them until the retention period ends (10 years by default).
CloudWatch, SNS, SES	Logs and alarms (emailed to your operations team) and the 'please sign' emails.

PART 4

What happens when someone signs

The same six steps happen whether you sign a document yourself or as part of an envelope.



Grey = in your browser Blue = QSign code Amber = AWS services

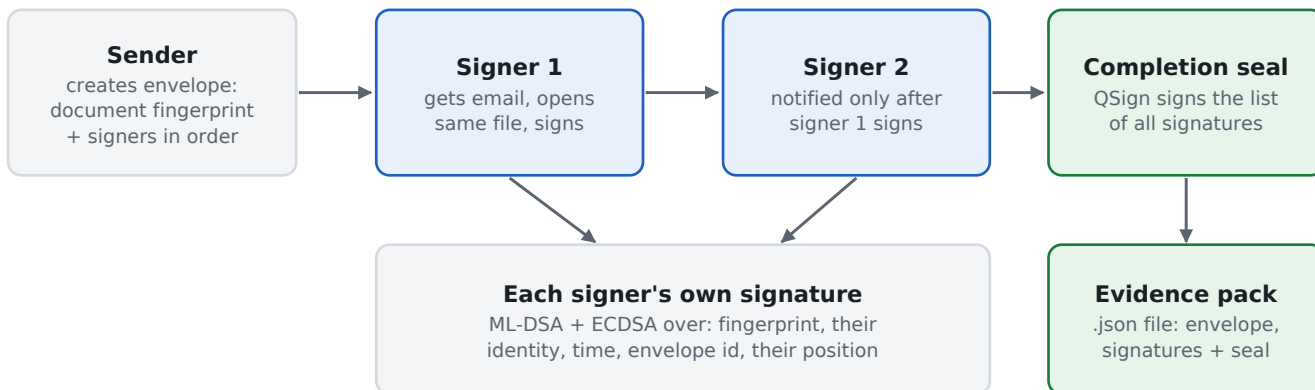
Step	What happens	Why it matters
1	The browser reads the file and computes its SHA-512 fingerprint.	The document never leaves the signer's computer. Privacy by design.
2	The signer ticks 'I have reviewed this document and I agree to sign it electronically'.	Laws in the USA, India and the EU require clear intent and consent. The exact sentence is signed too.
3	API Gateway (or QSign itself, when run as a container) checks the Cognito login token.	The signer's name and email come from the verified login, never from what is typed in the form.
4	QSign builds the manifest : fingerprint, file name and size, signer identity and organisation, time (UTC), reason, location, jurisdiction and, for envelopes, the envelope and signing order.	This small record is what actually gets signed. It ties the person to the exact file and moment.
5	QSign asks KMS to sign the manifest with the ML-DSA-65 key and with the ECDSA P-384 key.	The private keys never leave KMS. Every request is logged by AWS CloudTrail.
6	QSign writes an audit record, archives the signature in S3 Object Lock, and returns a .qsig.json file.	The signer keeps the file next to the document. The organisation keeps a copy nobody can tamper with.

Why a separate signature file?

QSign produces a detached signature (a small JSON file) instead of changing the document. It works for any file type (PDF, Word, images, code) and the original stays byte-for-byte unchanged. Embedding signatures inside PDFs (PAdES) is on the roadmap.

Envelopes: several signers, one proof

An envelope is how a company sends one document to several people and gets back a single proof that everyone signed it.



- **Create.** The sender picks the document (only its fingerprint is sent), lists up to 25 signers, chooses in order or any order, adds a message and an expiry date (1 to 365 days).
- **Invite.** Signers who have no account get a guest account automatically, with an email invitation. Guests can sign what was sent to them but cannot send envelopes.
- **Sign.** When it is their turn, each signer opens the envelope and selects their copy of the document. QSign accepts the signature only if the fingerprint matches the one in the envelope. They could not sign a different file even by mistake.
- **Decline or cancel.** A signer can decline with a reason; the sender or an organisation admin can cancel. Either stops the envelope.
- **Seal.** When the last person signs, QSign signs a list of every signer's signature (its own fingerprint, order, email and time). This **completion seal** is the lock on the envelope.
- **Evidence pack.** One JSON file containing the envelope, every signature and the seal. Hand it to an auditor, a court or the other party; they can verify it without an account.

What the seal prevents

Without a seal, someone could quietly drop a signer, add a fake signature, or swap the order. With it, any such change breaks the seal's ML-DSA and ECDSA signatures and verification fails.

Verifying a signature

Verification needs only the document and its signature file or evidence pack. No account, and no trust in QSign itself.

1 Signatures
ML-DSA and ECDSA
math both pass

2 Document
fingerprint of your
file matches

3 Keys
fingerprints are the
organisation's own

4 Seal (envelopes)
no signature added,
removed or swapped

Three ways to verify

Where	How
QSign web page	Open the Verify tab, choose the document and the .qsig.json or evidence file. The fingerprint is computed in the browser.
Free QSign Verifier	One open-source file anyone may use, no account or subscription: <code>python qsing_verify.py contract.pdf contract.pdf.qsig.json --trust <key fingerprint></code>
Independent tools	OpenSSL 3.5 or newer, or any library that supports ML-DSA. The repository README lists the exact commands.

What a failure means

Message	Meaning	What to do
Document does not match	The file is not the one that was signed, or it was changed afterwards.	Ask for the original file.
Signature is not valid	The signature file was edited, or was not made by those keys.	Treat as forged.
No valid post-quantum signature	Someone removed the ML-DSA part, leaving only the classical one.	Treat as forged.
Keys not trusted	The maths is fine, but the keys are not this organisation's published keys.	Check who really made it.
Not the one that was sealed	A signature in an evidence pack was swapped, added or removed.	Treat the pack as tampered.

What is inside a signature file

```
{
  "format": "qsing/1",
  "manifest": {
    "document": { "name": "msa.pdf", "size": 48213, "sha512": "9f86d0...alc3e7" },
    "signer": { "name": "Asha Rao", "email": "asha@acme.com", "org_id": "acme",
      "verified_by": "cognito:https://cognito-idp.ap-south-1..." },
    "intent": "I have reviewed this document and I intend to sign it electronically.",
    "jurisdiction": "IN", "signed_at": "2026-10-08T07:43:29Z",
    "envelope": { "id": "4ble...", "signer_order": 1, "total_signers": 2 }
  },
  "signatures": [
    { "alg": "ML-DSA-65", "key_fingerprint": "c41a...", "signature": "..." },
    { "alg": "ECDSA-P384-SHA384", "key_fingerprint": "7be2...", "signature": "..." }
  ]
}
```

Each signature also carries the public key it was made with, so it can be checked offline.

Security

Who can do what, and how QSign stops the attacks that matter for e-signatures.

Who can do what

Role	Can	Cannot
Anyone (no account)	Verify signatures and evidence packs; see the published key fingerprints.	Sign, see envelopes.
Guest (external signer)	Sign envelopes sent to them; view and verify them.	Send envelopes; sign their own documents; see other people's envelopes.
Member of an organisation	Everything a guest can, plus sign their own documents and send envelopes while the plan is active.	See envelopes they are not part of; invite users.
Organisation admin	Invite colleagues (and other admins); see all their organisation's envelopes and audit trail; cancel envelopes.	See other organisations' data. Change who signed what.
AWS account owner	Deploy, rotate keys, read logs.	Export the private keys. Delete or edit locked signatures before retention ends.

Attacks and defences

Attack	Defence
Steal the signing key	Keys exist only inside KMS hardware and cannot be exported. Only QSign's role may use them, and every use is logged in CloudTrail.
Sign as someone else	Identity comes only from a verified Cognito token. Passwords plus an authenticator app (MFA). Request bodies cannot set identity.
Change a document after signing	Its fingerprint changes, so verification fails.
Edit the signature file	Both ML-DSA and ECDSA signatures fail.
Future quantum computer breaks ECDSA	The ML-DSA signature still holds, and a signature is only valid if every part verifies.
Add, drop or reorder envelope signers	The completion seal no longer matches.
Sign the wrong file in an envelope	QSign rejects any fingerprint that differs from the envelope's.
Two people act at the same moment	Optimistic locking: the second change is rejected and must retry.
Insider deletes or rewrites history	S3 Object Lock (compliance mode) and DynamoDB deletion protection with point-in-time recovery.
Organisation A reads organisation B's data	Every request is scoped to the caller's organisation; others get 'not found'.
Forged login token (container mode)	QSign checks the token's RS256 signature, issuer, audience, type and expiry itself.
Test settings left on in production	QSign refuses to start in development mode with real KMS keys.

The law: USA, India, EU

Strong cryptography is necessary but not sufficient. Each region also cares about who vouches for the signer's identity.

	QSign today	Needed for the highest level
USA ESIGN Act, UETA	Valid for most business contracts. The law is technology-neutral: intent, consent, a link to the record and a retained record, all of which QSign provides.	Federal agencies: FedRAMP authorisation (e.g. AWS GovCloud) and identity proofing to NIST SP 800-63. Some documents (wills, court orders) are excluded from e-signing.
India IT Act 2000	Useful supporting evidence, but not a 'digital signature' or 'electronic signature' that gets automatic legal recognition.	Integrate a CCA-empanelled eSign Service Provider (Aadhaar eSign) or a licensed Certifying Authority. QSign then adds its ML-DSA signature on top for quantum safety.
European Union eIDAS / eIDAS 2	An advanced electronic signature (AdES) with MFA-verified signers and a full audit trail.	A qualified signature (QES) needs a Qualified Trust Service Provider on the EU Trusted List. Integrate a QTSP's remote signing service.

Why the hybrid design helps legally

Licensed providers in India and the EU still issue RSA and elliptic-curve certificates. QSign can carry their legally recognised classical signature together with its own ML-DSA signature: valid under today's rules, and protected against tomorrow's quantum attacks.

Data location: deploy in Mumbai (ap-south-1) or Hyderabad (ap-south-2) for Indian customers (DPDP Act 2023), Frankfurt (eu-central-1) for EU customers (GDPR), and a US region or GovCloud for US customers. The full analysis is in **docs/COMPLIANCE.md**.

Running it

From an empty AWS account to a customer's first signed envelope.

Step	Command or action
1 Deploy	<code>cd infra && sam build && sam deploy --guided --stack-name qsign</code> Use <code>ArchiveRetentionDays = 1</code> while testing; locked records cannot be deleted early.
2 Email	Verify your sending address in Amazon SES and connect Cognito to SES for invitation emails.
3 First customer	<code>scripts/create-org-admin.sh qsign acme admin@acme.com "Asha Admin"</code>
4 First sign-in	The admin opens the App URL, sets a password and scans the authenticator key.
5 Invite and send	The admin invites colleagues from the Admin tab; anyone can send an envelope.
6 Publish keys	Share the fingerprints from <code>/api/keys</code> on your website so customers and courts can confirm them.

Plans

Every organisation gets a **30-day free trial** from the first time a member signs in, then a paid **Business** or **Enterprise** plan. The operator records a payment with `qsign plan set acme --plan business --until 2027-12-31`. When a plan ends, the organisation can no longer sign or send, but envelopes already sent can be finished and everything signed can always be viewed and verified. Guest signers never need a plan.

Operating it

- **Alarms** email your team when signing errors, throttling or server errors appear.
- **Key rotation**: create new KMS keys, list the old fingerprints as retired, and old signatures stay trusted.
- **Backups**: DynamoDB point-in-time recovery (35 days) plus write-once S3 copies of every signature.
- **Containers**: the same code runs on ECS, EKS or on-premises with built-in token checking.

What comes next

Roadmap item	Benefit
Trusted timestamps (RFC 3161)	Independent proof of when a document was signed.
Signatures inside PDFs (PAdES)	Adobe Reader and similar tools show the signature directly.
India eSign / EU QTSP integration	Highest legal recognition in India and the EU.
ML-DSA certificates from AWS Private CA	Keys formally tied to your company name.
CloudFront + AWS WAF	Web application firewall for internet-facing deployments.

Source code, tests and detailed guides: [README.md](#), [docs/OPERATIONS.md](#), [docs/API.md](#), [docs/COMPLIANCE.md](#) and [SECURITY.md](#) in this repository. Regenerate this PDF with `python3 docs/explainer/build_pdf.py`.